

Подключение уведомлений клиентам: ВКонтакте, Telegram и Email

Полная актуальная инструкция по уведомлениям сотрудников находится в файле [Настройка уведомлений сотрудников](#). Этот файл описывает клиентский сценарий и будущий личный кабинет.

Эта инструкция описывает текущую реализацию уведомлений в системе «Символика», подключение внешних каналов и рекомендуемую схему будущего личного кабинета заказчика.

1. Что система отправляет сейчас

Автоматически отправляется только одно клиентское уведомление:

Заказ готов и находится в офисе.

Для отправки одновременно должны выполняться все условия:

1. статус заказа — **Готов** ;
2. способ получения — самовывоз из офиса;
3. офисный статус заказа — **В офисе** ;
4. все позиции заказа находятся в офисе;
5. у клиента или компании выбран основной канал уведомлений;
6. для выбранного канала указан получатель.

Сообщение содержит:

- номер заказа;
- позиции, количество, цену и сумму;
- общую сумму заказа;
- сколько оплачено;
- остаток к оплате;
- адрес и время работы офиса;
- сайт и ссылку на сообщество ВКонтакте, если они заполнены.

Повторно успешно отправленное уведомление по тому же заказу не отправляется. Результат и ошибка провайдера сохраняются в `symbolika_customer_notifications`.

Если в заказе выбраны и клиент, и компания, используется канал самого клиента. Настройки компании используются, только когда у клиента выбран вариант **Не отправлять**.

2. Где настраиваются уведомления

Общие данные офиса

Откройте:

Заполните:

- адрес офиса;
- режим работы;
- адрес сайта;
- ссылку на сообщество ВКонтакте.

Канал конкретного клиента

1. Откройте раздел `Клиенты` или `Компании`.
2. Откройте карточку клиента/компании.
3. Найдите блок `Уведомление о готовом заказе в офисе`.
4. Выберите основной канал.
5. Укажите Email, Telegram chat ID или VK peer ID.

Изменение настроек клиента после перехода заказа в готовность запускает повторную проверку. Если уведомление по заказу уже имеет статус `sent`, второй раз оно не отправится.

3. Безопасность перед подключением

Токены ботов и пароли SMTP равнозначны паролям. Их нельзя:

- отправлять в чатах;
- вставлять в инструкции и скриншоты;
- хранить в Git;
- записывать непосредственно в исходный код.

В текущем `docker-compose.yml` значение `SYMBOLIKA_VK_TOKEN` задано непосредственно в файле. Перед рабочим подключением нужно:

1. отозвать текущий ключ сообщества ВКонтакте и выпустить новый;
2. заменить строку в `docker-compose.yml` на:

```
SYMBOLIKA_VK_TOKEN: "${SYMBOLIKA_VK_TOKEN:-}"
```

1. заменить жестко заданный локальный публичный адрес на:

```
SYMBOLIKA_PUBLIC_URL: "${SYMBOLIKA_PUBLIC_URL:-http://localhost:8057}"
```

1. хранить реальные ключи и публичный адрес только в файле `symbolika_directus_clean_install/.env`;
2. убедиться, что `.env` не попадает в Git (корневой `.gitignore` уже исключает его).

Пример `.env` без реальных секретов:

```
# Публичный адрес системы на VPS
SYMBOLIKA_PUBLIC_URL=https://account.example.ru

# ВКонтакте
SYMBOLIKA_VK_TOKEN=

# Telegram
SYMBOLIKA_TELEGRAM_BOT_TOKEN=

# Email / SMTP
SYMBOLIKA_SMTP_HOST=
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=
SYMBOLIKA_SMTP_PASSWORD=
SYMBOLIKA_EMAIL_FROM="Символика <mail@example.ru>"
```

После изменения `.env` контейнер надо пересоздать: обычный `docker restart` не перечитывает переменные окружения.

```
Set-Location symbolika_directus_clean_install
docker compose up -d --force-recreate directus
docker logs --tail 100 symbolika-directus
```

4. Подключение сообщества ВКонтакте

4.1. Подготовить сообщество

1. Откройте управление нужным сообществом ВКонтакте.
2. Включите сообщения сообщества.
3. В разделе работы с API создайте ключ доступа сообщества с правом отправки сообщений.
4. Скопируйте новый ключ в `SYMBOLIKA_VK_TOKEN` файла `.env`.
5. Не используйте пользовательский токен личной страницы.

Система вызывает метод VK API `messages.send`, передавая ключ сообщества, `peer_id`, `random_id` и текст сообщения.

Официальная документация:

- <https://dev.vk.com/ru/api/community-messages/getting-started>
- <https://dev.vk.com/ru/method/messages.send>

4.2. Подписать клиента

Сообщество не должно начинать диалог с человеком без его действия. Сначала клиент должен:

1. открыть сообщество;

2. нажать Сообщение ;
3. отправить любое сообщение, например Хочу получать уведомления о готовности заказа .

После этого нужно узнать числовой `peer_id` диалога. Пока входящий VK Callback API в системе не реализован, ID определяется вручную через список диалогов сообщества или метод `messages.getConversations` .

Пример проверки из PowerShell (токен не вставляется в команду и не сохраняется в истории):

```
$env:SYMBOLIKA_VK_TOKEN = Read-Host "VK token"
$vkResult = Invoke-RestMethod -Method Post `
  -Uri "https://api.vk.com/method/messages.getConversations" `
  -Body @{
    access_token = $env:SYMBOLIKA_VK_TOKEN
    v = "5.199"
    count = 20
  }
$vkResult.response.items | ForEach-Object {
  [PSCustomObject]@{
    peer_id = $_.conversation.peer.id
    message = $_.last_message.text
  }
}
```

В карточке клиента укажите:

Основной канал: ВКонтакте
VK peer ID: полученное числовое значение

Для личного диалога `peer_id` обычно совпадает с числовым ID пользователя. Не подставляйте короткое имя страницы вида `vk.com/username` .

5. Подключение Telegram-бота

5.1. Создать бота

1. Откройте официальный бот `@BotFather` .
2. Выполните команду `/newbot` .
3. Задайте имя и username, заканчивающийся на `bot` .
4. Сохраните полученный токен в `SYMBOLIKA_TELEGRAM_BOT_TOKEN` файла `.env` .
5. При компрометации токена немедленно перевыпустите его через BotFather.

Официальная инструкция Telegram: <<https://core.telegram.org/bots/tutorial>>.

5.2. Подписать клиента

Telegram-бот не может первым написать пользователю. Клиент должен:

1. открыть ссылку вида `https://t.me/ИМЯ_БОТА`;
2. нажать `Запустить` или отправить `/start`.

После этого получите `chat.id` через `getUpdates`:

```
$env:SYMBOLIKA_TELEGRAM_BOT_TOKEN = Read-Host "Telegram bot token"
$tgUpdates = Invoke-RestMethod `
    -Uri "https://api.telegram.org/bot$env:SYMBOLIKA_TELEGRAM_BOT_TOKEN/getUpdates"
$tgUpdates.result | ForEach-Object {
    [PSCustomObject]@{
        chat_id = $_.message.chat.id
        username = $_.message.chat.username
        name = ($_.message.chat.first_name, $_.message.chat.last_name -join " ").Trim()
        message = $_.message.text
    }
}
```

В карточке клиента укажите:

Основной канал: Telegram
Telegram chat ID: полученное числовое значение

Если у бота уже настроен webhook, `getUpdates` использовать одновременно нельзя. Проверить webhook можно методом `getWebhookInfo`. Текущая реализация «Символики» использует Telegram только для исходящей отправки и сама webhook не устанавливает.

Официальные материалы:

- `<https://core.telegram.org/bots>`
- `<https://core.telegram.org/bots/api#getupdates>`

6. Подключение Email

Нужен почтовый ящик или SMTP-сервис, разрешающий отpravку через SMTP. У провайдера получите:

- SMTP host;
- порт;
- логин;
- пароль приложения или SMTP-пароль;
- допустимый адрес отправителя.

Для порта `465`:

```
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
```

Для порта `587` обычно используется:

```
SYMBOLIKA_SMTP_PORT=587
SYMBOLIKA_SMTP_SECURE=false
```

Точный режим уточняйте у своего почтового провайдера. В Nodemailer порт 465 использует TLS сразу, а для 587 соединение обычно повышается до TLS после подключения: <https://nodemailer.com/smtp>.

Для рабочей доставляемости писем настройте у домена:

- SPF;
- DKIM;
- DMARC;
- адрес отправителя на том же подтвержденном домене.

В карточке клиента укажите:

```
Основной канал: Email
Email для уведомления: адрес заказчика
```

Сейчас письмо отправляется как обычный текст, без HTML-шаблона.

7. Как проверить отправку

Проверку лучше проводить на отдельном тестовом клиенте и тестовом заказе.

1. Настройте канал тестового клиента.
2. Создайте заказ со способом получения `Выдача в офисе`.
3. Добавьте хотя бы одну позицию.
4. Доведите заказ и позиции до статуса `Готов`.
5. Переведите все позиции в офисный статус `В офисе`.
6. Проверьте получение сообщения.
7. Проверьте журнал `symbolika_customer_notifications` и логи Directus.

```
docker logs --tail 150 symbolika-directus | Select-String -Pattern "customer
notification|Telegram|VK|SMTP|send failed"
```

Основные причины ошибок:

Ошибка	Что проверить
Не настроен Telegram-бот	<code>SYMBOLIKA_TELEGRAM_BOT_TOKEN</code> и пересоздание контейнера
Telegram chat not found	клиент запустил бота, chat ID указан без ошибок
Не настроена отправка ВКонтакте	ключ сообщества находится в окружении контейнера
VK отказал в отправке	сообщения сообщества включены, клиент начал диалог, указан правильный peer ID

Ошибка	Что проверить
Не настроена отправка email	host, user, password и from заполнены
SMTP authentication failed	пароль приложения, разрешение SMTP и параметры порта/TLS
Для выбранного канала не указан получатель	Email, chat ID, peer ID или телефон заполнены в карточке
Записи в журнале нет	заказ не удовлетворяет всем условиям готовности и нахождения в офисе
Статус <code>sent</code> , но повторного сообщения нет	защита от повторной отправки сработала штатно

Не выводите значения токенов командами `docker inspect`, `docker compose config` или в общие журналы/скриншоты.

8. Как клиент оформляет подписку сейчас

Самостоятельной подписки клиента пока нет. Рабочий процесс сейчас такой:

1. менеджер получает согласие клиента на выбранный канал;
2. клиент пишет сообществу ВКонтакте или запускает Telegram-бота;
3. менеджер получает peer ID/chat ID;
4. менеджер открывает карточку клиента или компании;
5. выбирает основной канал и вводит получателя;
6. при просьбе клиента отключить уведомления выбирает `Не отправлять`.

Согласие желательно фиксировать датой, источником и сотрудником, который его получил. Текущая модель хранит только выбранный канал и адрес получателя, но не хранит отдельный журнал согласий.

9. Рекомендуемый личный кабинет заказчика

Личный кабинет стоит делать отдельным внешним модулем. Не следует выдавать клиентам доступ в стандартную админку Directus или внутренние рабочие модули.

Что клиент должен видеть

- текущие и архивные заказы;
- позиции, количество, цены и публичные статусы;
- сроки и способ получения;
- сумму заказа, оплаты, остаток и переплату;
- общую сверку взаиморасчетов;
- контактные данные менеджера;
- публичные макеты/файлы, если они специально разрешены клиенту;
- настройки уведомлений и журнал отправленных сообщений.

Что клиент не должен видеть

- себестоимость и прибыль;
- контрагентов и закупочные цены;
- внутренние производственные комментарии;
- задачи сотрудников;
- внутреннюю ленту событий;
- служебные статусы и автоматизации;
- данные других клиентов и компаний.

Как оформить подписку безопасно

Email

1. Клиент вводит адрес.
2. Система отправляет ссылку подтверждения с одноразовым токеном.
3. Канал включается только после перехода по ссылке.

Telegram

1. Кабинет показывает кнопку Подключить Telegram .
2. Клиент открывает ссылку `https://t.me/ИМЯ_БОТА?start=ОДНОРАЗОВЫЙ_ТОКЕН` .
3. Telegram webhook получает `/start` и связывает `chat_id` с нужным клиентом.
4. В кабинете появляется статус Подключено .

ВКонтакте

1. Кабинет выдает одноразовый код или ссылку.
2. Клиент пишет этот код в сообщения сообщества.
3. VK Callback API принимает сообщение и связывает `peer_id` с клиентом.
4. В кабинете появляется статус Подключено .

Для VK и Telegram текущей исходящей отправки недостаточно: потребуется защищенный webhook для приема сообщений и таблица одноразовых токенов привязки.

Рекомендуемая модель данных

Не хранить подписку только в полях клиента. Добавить отдельную коллекцию, например `customer_notification_subscriptions` :


```
id
customer / customer_company
channel
recipient
status: pending / confirmed / disabled / failed
verification_token_hash
verification_expires_at
confirmed_at
disabled_at
consent_source
consent_ip
created_at
updated_at
```

Это позволит:

- подключить несколько каналов;
- выбрать один основной;
- хранить подтверждение согласия;
- безопасно отвязать старый Telegram/VK аккаунт;
- не перезаписывать контактные данные клиента;
- вести аудит подписок.

Доступ к кабинету

Рекомендуемый первый вариант — вход по одноразовой ссылке на Email. Для компаний позже можно добавить несколько пользователей и роли:

- владелец компании;
- бухгалтер;
- сотрудник, оформляющий заказы;
- только просмотр.

Ссылки должны быть короткоживущими и одноразовыми. Нельзя использовать постоянный `public_token` позиции как авторизацию ко всему кабинету.

10. Рекомендуемый порядок реализации

Этап 1 — довести текущие уведомления

- вынести все секреты в `.env` ;
- добавить кнопку `Отправить тестовое уведомление` ;
- добавить понятный журнал отправок в рабочий модуль;
- добавить кнопку повторной отправки для статуса `failed` ;
- добавить проверку доступности канала до сохранения.

Этап 2 — страница самостоятельной подписки

- отдельная публичная страница без полного личного кабинета;
- подтверждение Email;
- Telegram deep link и webhook;
- VK Callback API и одноразовый код;
- отключение подписки.

Этап 3 — полноценный кабинет заказчика

- заказы и позиции;
- финансы и сверка;
- файлы;
- менеджер и контакты;
- настройки уведомлений;
- доступ нескольких сотрудников компании.

Такой порядок быстрее даст безопасную самостоятельную подписку и не заставит сразу строить весь клиентский портал.